

COMPTE-RENDU TECHNIQUE

Analyse de conversations patient-thérapeute sur la santé mentale

Auteur : LARROUX Logan

Encadrante : Lynda Tamine-Lechani

Équipe :

*Ulysse Chasseigne, Idir Yahiaoui, Bantsoukissa Laure,
Cruz Fernandes Diogo, Emin Belkheir, Ezzo-Y-N Trésor PANA,
Logan Larroux, Yvan Sellier, Victor Blanquart-Blanchet, Lucas Da Silva*

12 mai 2026



[Info5.BE-SHS]

Données : [Kaggle](#) — NLP Mental Health Conversations

Code : [GitHub](#) — branche Logan

Table des matières

1	Contexte et organisation	2
1.1	Différence entre méthode supervisée et non-supervisée	2
1.2	Métriques d'évaluation utilisées	2
1.2.1	Métriques communes aux deux méthodes	3
1.2.2	Métriques propres à la méthode supervisée	3
1.2.3	Métriques propres à la méthode non-supervisée	4
1.2.4	Tableau récapitulatif	5
2	Résultats par membre	6
2.1	Méthode supervisée — Naive Bayes et Logistic Regression	6
2.1.1	Diogo et Ulysse — Naive Bayes BoW et TF-IDF	6
2.1.2	Logan — Naive Bayes + Logistic Regression	9
2.2	Méthode non-supervisée — K-Means (Emin et Victor)	12
2.2.1	Victor — Implémentation Word2Vec	13
2.2.2	Emin — Comparaison TF-IDF et robustesse du pipeline	14
2.2.3	Synthèse — Comparaison TF-IDF vs Word2Vec pour K-Means . . .	16
3	Comparaison globale : supervisé vs. non-supervisé	18
4	Décision finale : modèle transmis au Groupe 2	19
5	Points de vigilance et perspectives	19

1 Contexte et organisation

L'étape 2 a divisé le groupe 1 en deux sous-groupes travaillant en parallèle sur le même jeu de données d'entraînement (`combined_data.csv`, *Sentiment Analysis for Mental Health* sur Kaggle), avec pour objectif commun d'annoter automatiquement le dataset principal `train.csv` en vue de sa transmission au Groupe 2.

Sous-groupe	Méthode	Membres
Supervisé	Naive Bayes (BoW + TF-IDF) + Logistic Regression (TF-IDF)	Diogo, Ulysse, Log
Non-supervisé	K-Means (Word2Vec + TF-IDF)	Emin, Victor

Le pipeline commun aux deux sous-groupes comprend : nettoyage du texte, tokenisation (NLTK + Porter Stemmer), et vectorisation (BoW, TF-IDF, Word2Vec). Le split des données est identique pour les deux : **60 % train / 20 % validation / 20 % test**, avec stratification sur les classes.

1.1 Différence entre méthode supervisée et non-supervisée

Une méthode supervisée apprend à partir d'**exemples déjà étiquetés**. Pour chaque entrée, on connaît la bonne réponse pendant l'entraînement ; le modèle apprend donc à associer des entrées à des sorties connues.

Exemple : classer des e-mails en spam / non-spam, prédire un prix, détecter une maladie à partir de symptômes.

Une méthode non-supervisée, quant à elle, apprend à partir de **données non étiquetées**. Il n'existe pas de bonne réponse fournie au préalable ; le but est de découvrir des structures cachées, des groupes ou des patterns dans les données.

Exemple : regrouper des clients en segments, détecter des anomalies, réduire la dimension des données.

En résumé :

- Supervisée → on connaît la réponse attendue.
- Non-supervisée → on ne connaît pas la réponse, on cherche des structures.

1.2 Métriques d'évaluation utilisées

Afin de comparer les modèles et sélectionner le meilleur pour annoter `train.csv`, nous distinguons trois catégories de métriques selon leur domaine d'application.

1.2.1 Métriques communes aux deux méthodes

Ces deux métriques s'appliquent aussi bien à la méthode supervisée qu'à la méthode non-supervisée (après association des clusters aux labels pour cette dernière), ce qui en fait les seuls indicateurs permettant une **comparaison directe** entre les deux approches.

Accuracy Pourcentage de sentiments correctement prédits :

$$\text{Accuracy} = \frac{\text{Nb. de bonnes prédictions}}{\text{Nb. total d'exemples}}$$

Matrice de confusion Permet de visualiser les prédictions par rapport aux vraies classes. Les lignes représentent les vraies classes, les colonnes les classes prédites. Elle révèle les erreurs systématiques du modèle : quelles classes sont confondues entre elles, et dans quelle direction.

1.2.2 Métriques propres à la méthode supervisée

Ces métriques nécessitent de connaître les vrais labels pour chaque exemple, ce qui est possible en apprentissage supervisé mais pas directement applicable au clustering non-supervisé.

MSE (Mean Squared Error) Erreur moyenne au carré entre la vraie valeur et la valeur prédite :

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

R² (Coefficient de détermination) Mesure la proportion de variance des données expliquée par le modèle :

$$R^2 = 1 - \frac{\text{Erreur modèle}}{\text{Erreur moyenne}}$$

Attention — MSE et R² sont des métriques de régression. Elles supposent qu'il existe un ordre et une distance numérique significative entre les valeurs prédites et les valeurs réelles. Or nos labels sont nominaux (**Depression, Anxiety, Normal...**) encodés arbitrairement en entiers : la "distance" entre deux classes n'a aucun sens clinique ni mathématique. Ces deux métriques ne sont donc **pas des indicateurs fiables** pour ce problème et ne doivent pas être utilisées comme critères de décision. L'accuracy et le F1-score sont à privilégier.

Precision, Recall et F1-score Ces trois métriques issues du classification report permettent d'évaluer les performances **classe par classe**, ce qui est essentiel ici en raison du fort déséquilibre entre les 7 catégories de sentiments.

- **Precision** — parmi les prédictions positives, combien sont vraiment positives ?

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall** — parmi les vrais positifs, combien ont été retrouvés ?

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-score** — compromis entre precision et recall, particulièrement utile sur des classes déséquilibrées :

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Avec : TP = vrai positif, TN = vrai négatif, FP = faux positif, FN = faux négatif.

Pour comparer les méthodes supervisées entre elles (NB vs LR), nous privilégions le **F1-score macro** (moyenne non pondérée sur les classes) plutôt que le F1 *weighted*, afin de ne pas masquer les mauvaises performances sur les classes minoritaires (*Stress*, *Personality disorder*).

1.2.3 Métriques propres à la méthode non-supervisée

Ces métriques sont spécifiques au clustering : elles évaluent la qualité des groupes formés **sans faire référence aux vrais labels** pendant l'entraînement, ce qui les rend non comparables directement avec les métriques supervisées.

Purity globale Mesure la cohérence interne des clusters : pour chaque cluster, on retient uniquement le label majoritaire parmi ses membres.

$$\text{Purity} = \frac{1}{N} \sum_k \max_j |C_k \cap L_j|$$

Avec N le nombre total d'exemples, C_k le cluster k et L_j la classe réelle j .

Plus la Purity est proche de 1, meilleur est le regroupement. Attention : si chaque point forme son propre cluster, la purity vaut 1 — cette mesure peut donc être trompeuse si utilisée seule.

ARI (Adjusted Rand Index) Mesure la similarité entre les vrais labels et les clusters trouvés, en corrigeant le score pour tenir compte du hasard. Sa valeur est comprise entre -1 (pire qu'aléatoire) et 1 (clustering parfait). Contrairement à la Purity, l'ARI **pénalise un trop grand nombre de clusters** et les mauvais regroupements, ce qui en fait un indicateur plus robuste.

1.2.4 Tableau récapitulatif

Catégorie	Mesure	Supervisée	Non-supervisée
Communes	Accuracy	Oui	Oui (après mapping)
	Matrice confusion	Oui	Oui
Supervisée uniquement	MSE	Oui	Non
	R^2	Oui	Non
	Precision	Oui	Non
	Recall	Oui	Non
	F1-score	Oui	Non
Non-supervisée uniquement	Purity	Non	Oui
	ARI	Non	Oui

2 Résultats par membre

2.1 Méthode supervisée — Naive Bayes et Logistic Regression

Qu'est-ce que Naive Bayes ?

Naive Bayes répond à la question : « **Étant donné les mots de ce texte, quelle est la catégorie la plus probable ?** »

C'est un classificateur probabiliste : plutôt que de tracer une frontière entre les classes comme le ferait un SVM, il **calcule une probabilité** pour chaque classe et retourne celle qui est la plus élevée, en s'appuyant sur le théorème de Bayes :

$$P(\text{classe} \mid \text{mots}) \propto P(\text{classe}) \times \prod_i P(\text{mot}_i \mid \text{classe})$$

Le modèle fait une hypothèse simplificatrice : chaque mot est supposé **indépendant** des autres. Cette simplification rend l'algorithme très rapide à entraîner et étonnamment efficace sur les textes malgré son aspect « naïf ».

Qu'est-ce que la Logistic Regression ?

La Régression Logistique répond à la question : « **Étant donné ce vecteur TF-IDF, à quelle classe ce texte appartient-il le plus probablement ?** »

Contrairement à Naive Bayes qui calcule des probabilités indépendamment mot par mot, la Régression Logistique **apprend un poids pour chaque feature** (ici chaque n -gram TF-IDF) et combine ces poids pour produire un score par classe.

	Naive Bayes	Logistic Regression
Hypothèse	Mots indépendants	Aucune hypothèse sur les features
Frontière	Probabiliste naïve	Frontière de décision optimisée
Classes déséquilibrées	Biais vers la classe majoritaire	Géré via <code>class_weight='balanced'</code>
Corrélations entre mots	Ignorées	Prises en compte implicitement

Le paramètre clé de la LR est **C** (inverse de la régularisation) : un **C** petit entraîne une régularisation forte (modèle simple, évite l'overfitting) ; un **C** grand entraîne une régularisation faible (modèle plus complexe, peut overfit).

2.1.1 Diogo et Ulysse — Naive Bayes BoW et TF-IDF

Diogo et Ulysse ont produit les résultats de référence pour la méthode supervisée.

Tuning de l'hyperparamètre alpha (lissage de Laplace) :

- Valeurs testées : [0.001, 0.01, 0.05, 0.1, 0.5, 1.0, 2.0, 5.0]
- Meilleur alpha BoW : **0.1** (accuracy validation : 0.6606)
- Meilleur alpha TF-IDF : **0.05** (accuracy validation : 0.6761)

Résultats sur l'ensemble test :

Modèle	Accuracy test
Naive Bayes + BoW	0.6666
Naive Bayes + TF-IDF	0.6875

Classification report (NB + BoW) :

Classe	Precision	Recall	F1-score
Anxiety	0.68	0.73	0.71
Bipolar	0.60	0.76	0.68
Depression	0.57	0.61	0.59
Normal	0.92	0.70	0.80
Personality disorder	0.50	0.56	0.53
Stress	0.52	0.43	0.47
Suicidal	0.60	0.72	0.65

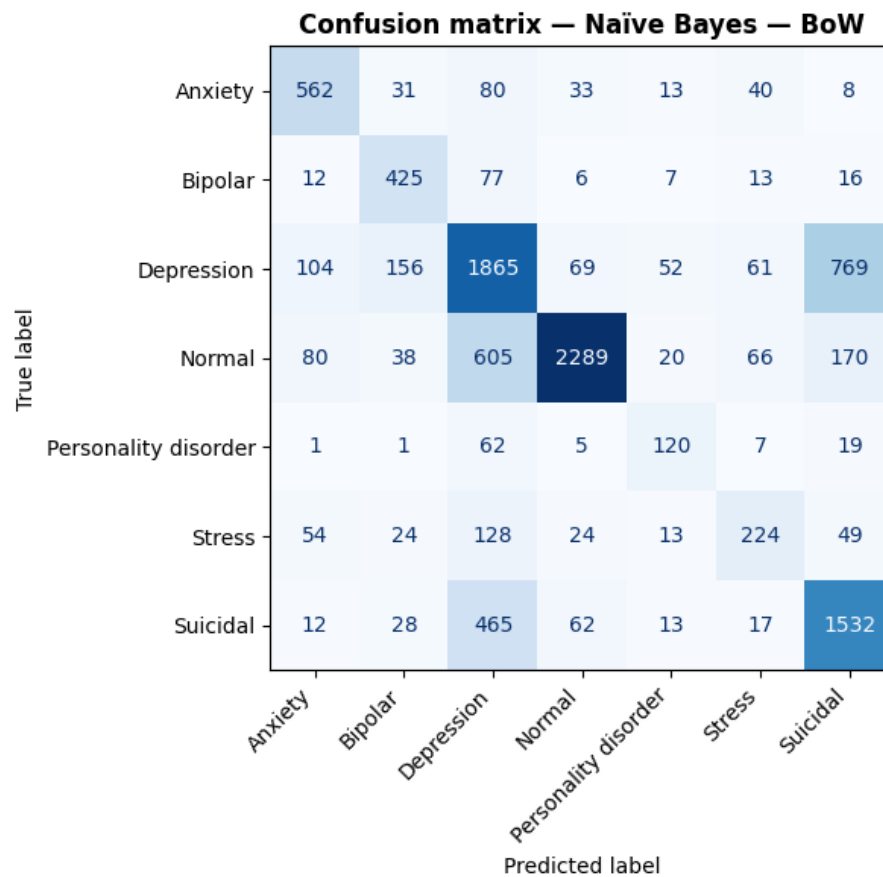


FIGURE 1 – Matrice de confusion — NB + BoW (Diogo et Ulysse)

Classification report (NB + TF-IDF) :

Classe	Precision	Recall	F1-score
Anxiety	0.76	0.71	0.73
Bipolar	0.82	0.58	0.68
Depression	0.54	0.77	0.63
Normal	0.90	0.77	0.83
Personality disorder	0.97	0.29	0.45
Stress	0.83	0.27	0.41
Suicidal	0.65	0.60	0.62

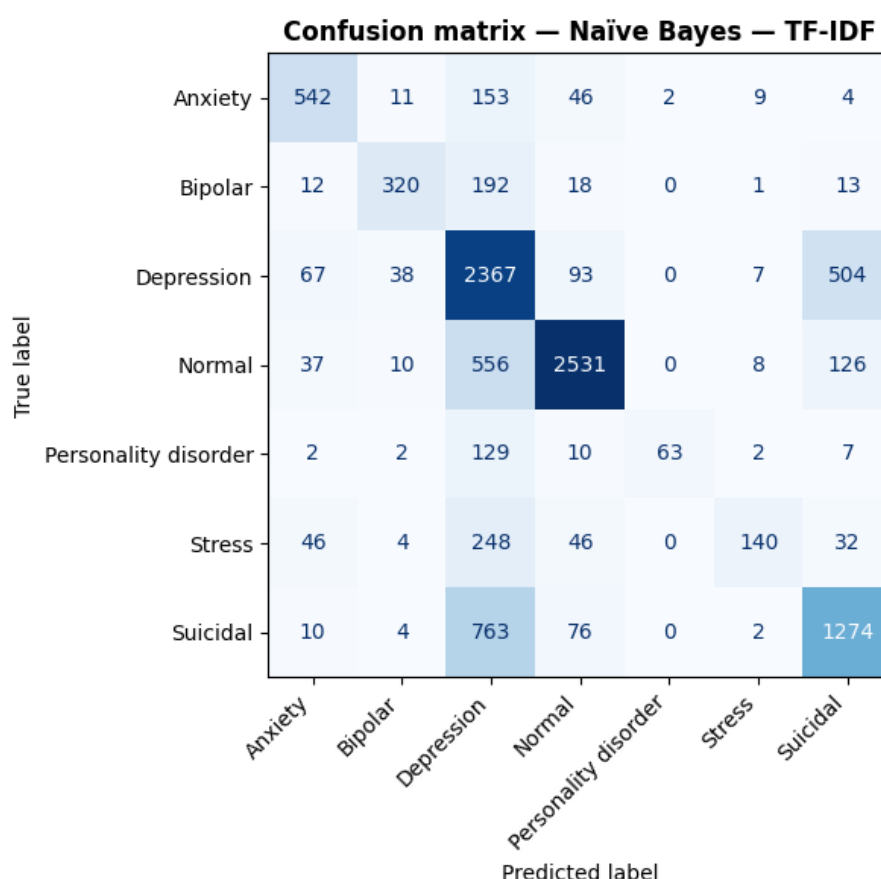


FIGURE 2 — Matrice de confusion — NB + TF-IDF (Diogo et Ulysse)

TF-IDF est meilleur sur les deux classes les plus représentées (*Normal* et *Depression*), ce qui tire son accuracy globale vers le haut. Mais sur les 5 autres classes, **BoW gagne à chaque fois**, parfois très largement : *Stress* (43 % vs 27 % de recall) ou *Bipolar* (76 % vs 58 % de recall).

Pourquoi ? TF-IDF pénalise les mots fréquents. C'est utile pour séparer *Normal* de *Depression*, mais pour les classes rares (*Stress*, *Personality disorder*), les mots caractéristiques sont précisément des mots fréquents dans le corpus global (*overwhelmed*, *pressure*, *relationship*...). TF-IDF les « dépouille » de leur signal. Le cas de *Personality disorder* est frappant : **precision 0.97 mais recall 0.29** — le modèle est très sûr quand il prédit cette classe, mais il rate 71 % des vrais cas.

2.1.2 Logan — Naive Bayes + Logistic Regression

Logan a produit le notebook de base structurant le travail commun. Sa contribution individuelle principale est l'implémentation d'une **Régression Logistique** (LR + TF-IDF optimisé), identifiée comme « version extra-performante ».

Paramétrage TF-IDF amélioré :

```
tfidf_vectorizer = TfidfVectorizer(
    ngram_range=(1, 3),      # unigrammes, bigrammes et trigrammes
    max_features=50000,      # limite aux 50k features les plus
                             utiles
    sublinear_tf=True,       # log(tf) au lieu de tf
    min_df=2,                # ignore les mots < 2 documents
    max_df=0.95              # ignore les mots > 95% des documents
)
```

Tuning de l'hyperparamètre α (Naive Bayes) :

- Valeurs testées : [0.001, 0.005, 0.01, 0.03, 0.05, 0.07, 0.1]
- Meilleur α BoW : **0.1** (accuracy : 0.6606)
- Meilleur α TF-IDF : **0.07** (accuracy : 0.6816)

Tuning de l'hyperparamètre C (Logistic Regression) :

- Valeurs testées : [0.01, 0.05, 0.1, 0.3, 0.5, 1.0, 3.0, 5.0, 10.0]
- Meilleur C : **3.0** (accuracy validation : 0.7457)

Résultats comparés sur l'ensemble test :

Modèle	Accuracy test
Naive Bayes + BoW	0.6666
Naive Bayes + TF-IDF	0.6927
Logistic Regression + TF-IDF	0.7532

Comparaison F1-score NB + TF-IDF vs LR + TF-IDF :

Classe	F1 NB TF-IDF	F1 LR TF-IDF	Évolution
Anxiety	0.73	0.80	+0.07
Bipolar	0.67	0.79	+0.16
Depression	0.63	0.69	+0.06
Normal	0.85	0.90	+0.15
Personality disorder	0.40	0.66	+0.26
Stress	0.37	0.57	+0.20
Suicidal	0.63	0.64	+0.01

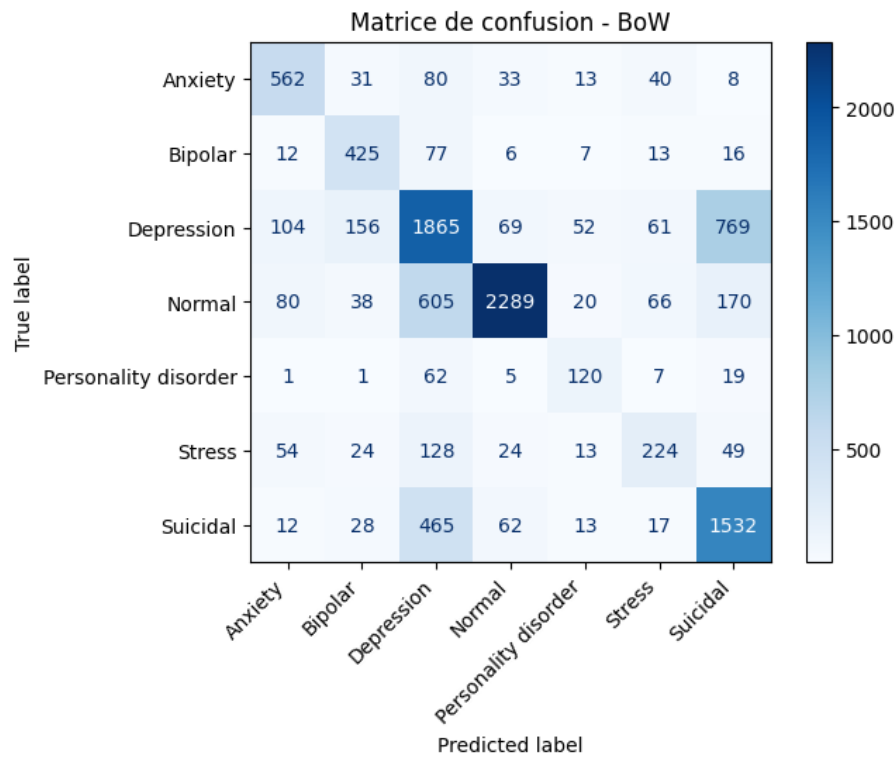


FIGURE 3 – Matrice de confusion — NB + BoW (Logan)

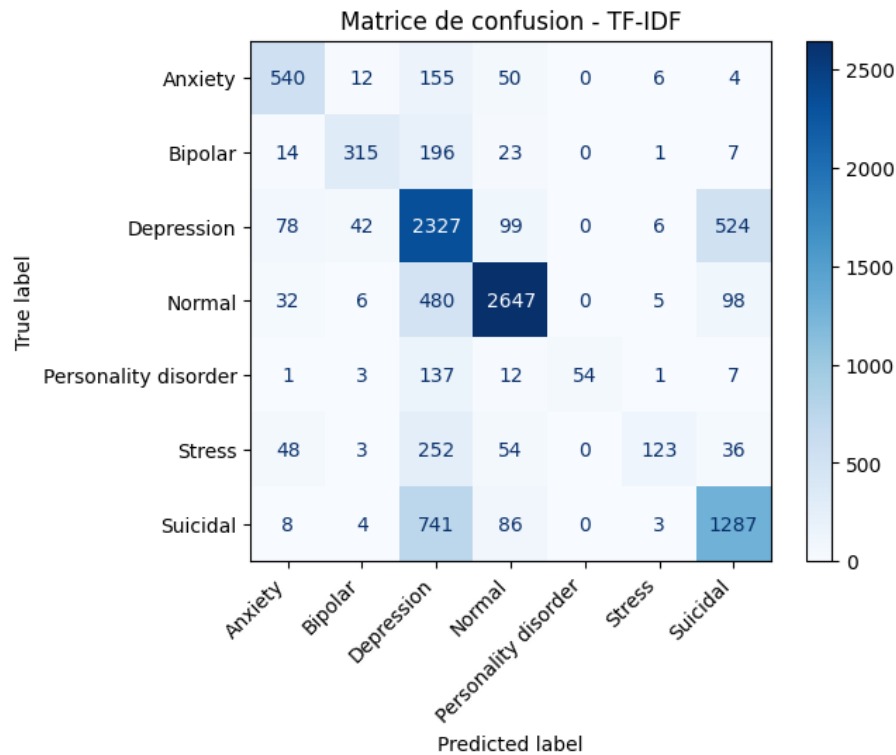


FIGURE 4 – Matrice de confusion — NB + TF-IDF (Logan)

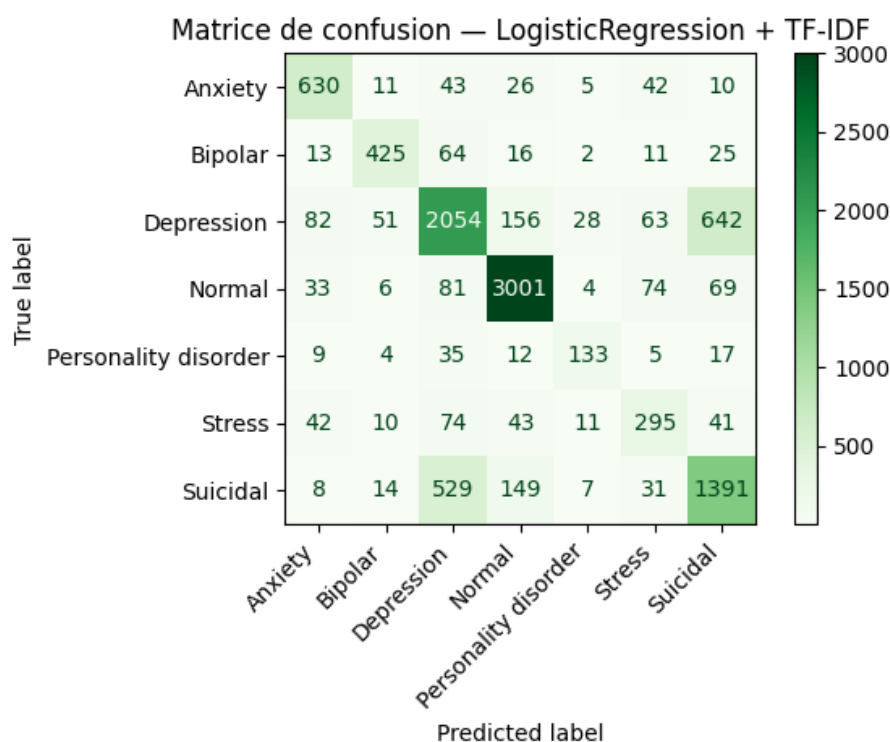


FIGURE 5 – Matrice de confusion — LR + TF-IDF (Logan)

La Logistic Regression améliore le F1-score sur **toutes les classes**, en particulier *Personality disorder* (+0.26) et *Stress* (+0.20) grâce au paramètre `class_weight='balanced'` qui compense le déséquilibre du dataset.

Problème critique — classe *Suicidal* : sur 2 129 vrais cas *Suicidal*, 1 391 sont correctement classés **mais 529 sont prédits *Depression***. Confondre *Suicidal* avec *Depression* peut avoir des conséquences réelles dans un contexte médical. Ce problème est commun à toutes les approches testées.

2.2 Méthode non-supervisée — K-Means (Emin et Victor)

Qu'est-ce que K-Means ?

L'algorithme K-Means regroupe les textes en calculant la **proximité mathématique** entre les vecteurs. Il place des centres (centroïdes) de manière aléatoire, puis les déplace itérativement jusqu'à ce que chaque texte soit rattaché au groupe le plus proche, minimisant ainsi l'inertie totale.

Le sous-groupe non-supervisé a comparé deux approches de vectorisation : **TF-IDF + SVD** et **Word2Vec**.

2.2.1 Victor — Implémentation Word2Vec

Victor a privilégié l'utilisation de **Word2Vec** pour capturer le sens profond des mots. Contrairement au TF-IDF qui ne fait que compter, Word2Vec comprend que des termes comme *triste* et *déprimé* sont sémantiquement proches.

Sélection de k : le nombre de groupes (entre 2 et 15) a été choisi automatiquement via le **score de silhouette cosinus** sur la validation, avec un plancher à `max(best_k, y_train.nunique())`.

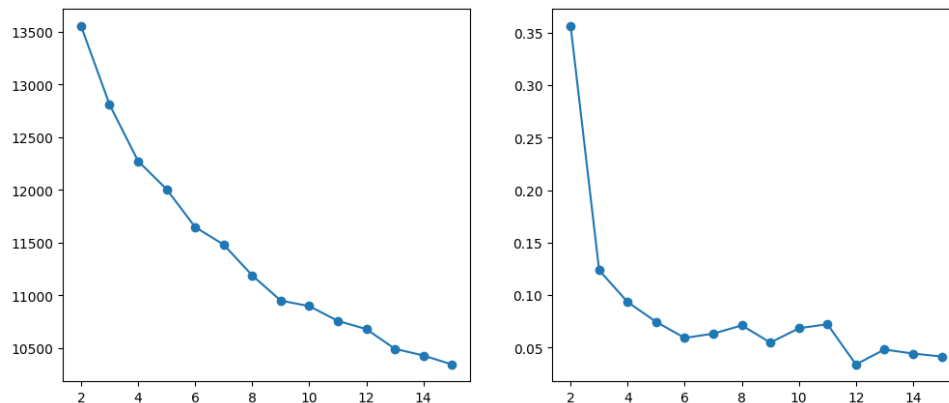


FIGURE 6 – Courbes WCSS et silhouette pour la sélection de k (Victor)

Mapping cluster $\rightarrow$ label (vote majoritaire) : pour nommer chaque cluster, on identifie la classe réelle dominante parmi ses membres. En cas de conflit, le cluster de plus haute pureté conserve l'étiquette (stratégie gloutonne).

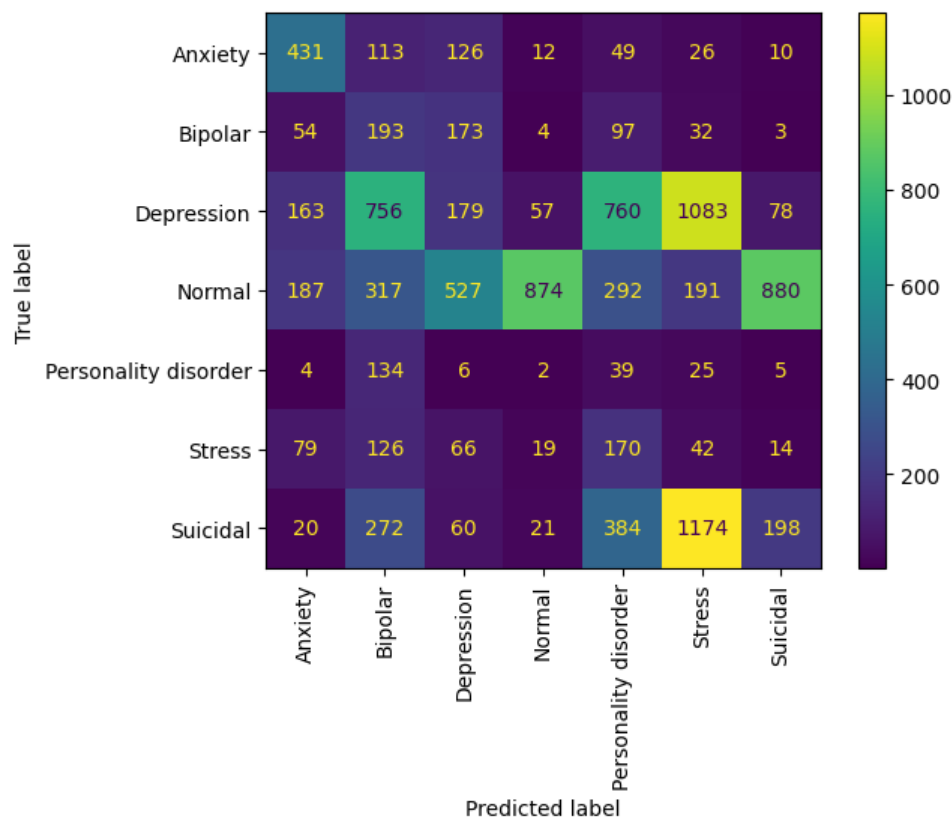


FIGURE 7 – Résultat du mapping et pureté des clusters (Victor)

Victor a corrigé une erreur du template qui tentait de mélanger TF-IDF et Word2Vec dans la cellule de déploiement final, ce qui rendait le pipeline incohérent.

2.2.2 Emin — Comparaison TF-IDF et robustesse du pipeline

Emin a complété l'étude en implémentant le pipeline complet sous **TF-IDF + SVD** pour comparer les deux représentations.

```
from sklearn.decomposition import TruncatedSVD
from sklearn.preprocessing import normalize

svd = TruncatedSVD(n_components=100, random_state=42)
X_train_tfidf_svd = svd.fit_transform(X_train_tfidf)
X_val_tfidf_svd = svd.transform(X_val_tfidf)
X_test_tfidf_svd = svd.transform(X_test_tfidf)

X_train_tfidf_norm = normalize(X_train_tfidf_svd, norm='l2')
X_val_tfidf_norm = normalize(X_val_tfidf_svd, norm='l2')
X_test_tfidf_norm = normalize(X_test_tfidf_svd, norm='l2')
```

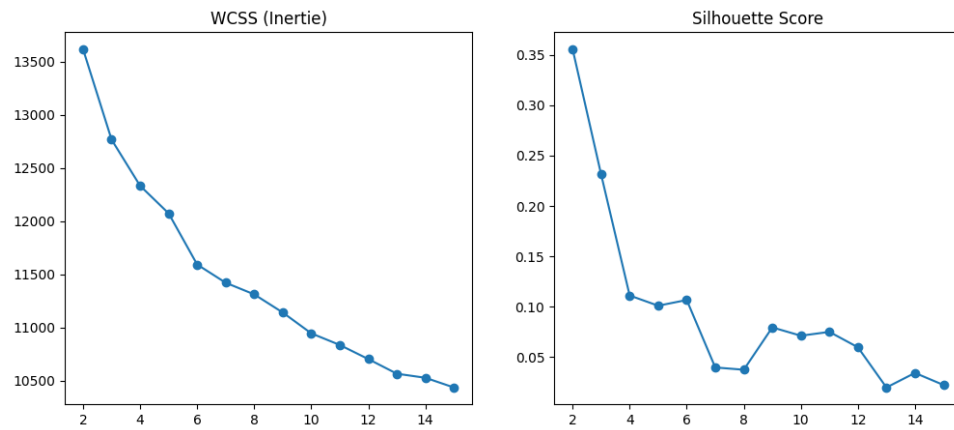
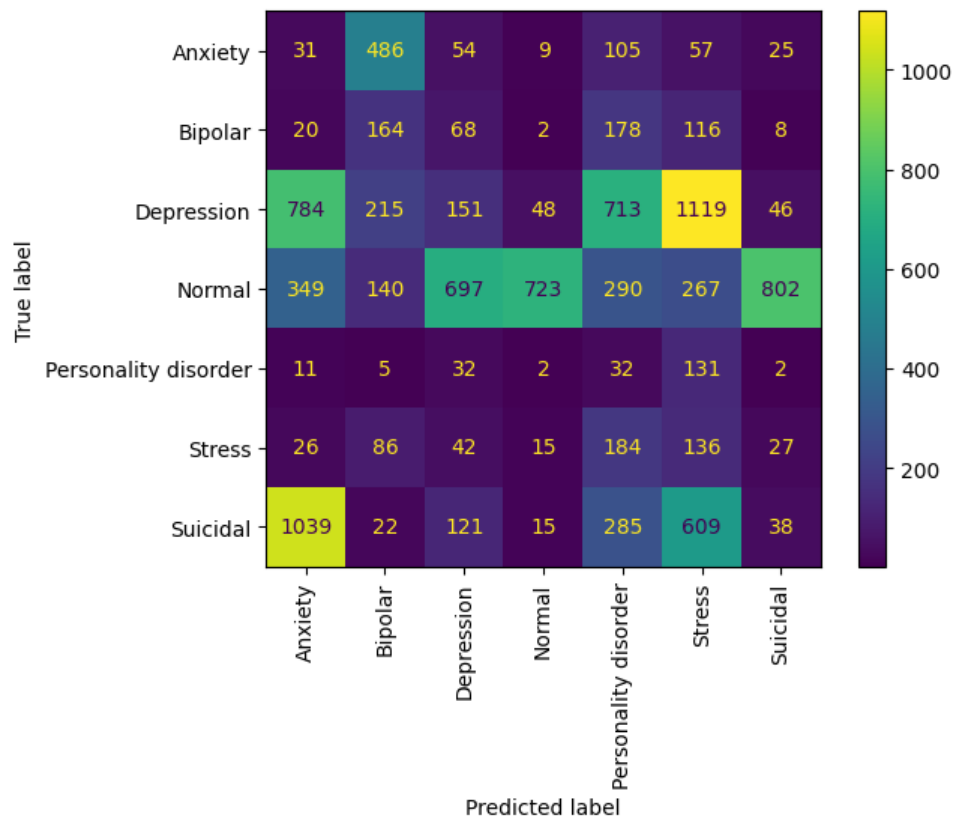
FIGURE 8 – Courbes WCSS et silhouette pour la sélection de k (Emin)

FIGURE 9 – Résultat du mapping et pureté des clusters (Emin)

Emin a également résolu des conflits de versions entre `gensim` et `scipy` dans l'environnement de travail, et standardisé la cellule de déploiement B.6 pour utiliser le modèle TF-IDF.

2.2.3 Synthèse — Comparaison TF-IDF vs Word2Vec pour K-Means

Critère	K-Means + TF-IDF	K-Means + Word2Vec
Meilleur k retenu	7	7
Accuracy test	0.4636	0.20
Purity globale	0.54	0.51
ARI	0.25	0.13
Besoin de SVD ?	Oui	Non
Dimension de l'espace	100 (après SVD)	100 (natif)
Pureté max cluster	0.85 (<i>Depression</i>)	0.90 (<i>Normal</i>)
Pureté min cluster	0.30 (<i>Personality disorder</i>)	0.30 (<i>Bipolar</i>)

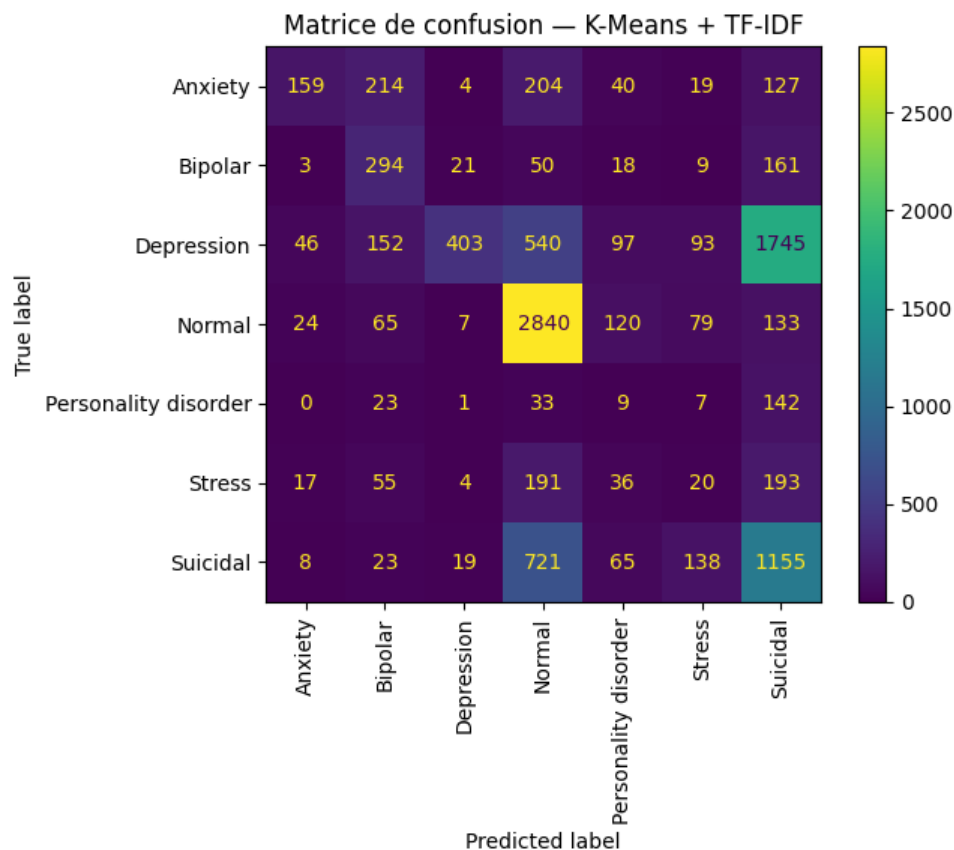


FIGURE 10 – Matrice de confusion — K-Means final (TF-IDF vs Word2Vec)

Contra-intuitivement, TF-IDF (46 %) surpasse Word2Vec (20 %). Les catégories cliniques du dataset (*Depression*, *Anxiety*, etc.) se reconnaissent avant tout par un **vocabulaire symptomatique spécifique**, ce que TF-IDF met précisément en avant. Word2Vec, qui regroupe les mots par contexte sémantique large, tend à fusionner

Depression et *Suicidal* dans les mêmes clusters — le mapping glouton force alors ces gros clusters sur des labels minoritaires, effondrant l'accuracy globale.

Modèle retenu pour le non-supervisé : K-Means + TF-IDF.

Fichier produit : `train_with_sentiments_kmeans.csv`.

3 Comparaison globale : supervisé vs. non-supervisé

Critère	NB TF-IDF (supervisé)	K-Means TF-IDF (non-supervisé)
Accuracy test	0.6875	0.4636
Macro avg F1	0.62	—
Weighted avg F1	0.69	—
Voit les labels à l'entraînement ?	Oui	Non
Hyperparamètre clé	alpha = 0.05	k = 7
Fichier annoté	train_with_sentiments.csv	train_with_sentiments_kmeans

L'écart de **+22 points d'accuracy** en faveur du supervisé est cohérent et attendu : le supervisé a accès aux vrais labels pendant l'entraînement, le non-supervisé pas. Les deux méthodes utilisent la même représentation TF-IDF, donc l'écart reflète purement la différence supervisé/non-supervisé.

Les mêmes classes sont difficiles pour les deux approches (*Personality disorder*, *Stress*, confusion *Depression/Suicidal*). Cela pointe vers une **limite des données elles-mêmes** (classes sous-représentées, vocabulaire qui se chevauche) plutôt qu'un problème algorithmique.

4 Décision finale : modèle transmis au Groupe 2

→ Modèle retenu : Naive Bayes + TF-IDF

Critère	Justification
Accuracy 68.75 %	Meilleure performance absolue parmi tous les modèles testés
Annotations fiables	Les labels transmis au Groupe 2 doivent être les plus corrects possible pour que le filtrage par sentiments soit utile
Modèle léger	NB est rapide à déployer et à sérialiser (.pkl)

Fichier à transmettre au Groupe 2 : `train_with_sentiments.csv`

Colonne d'annotation : `predicted_sentiment`

Classes prédites : Anxiety, Bipolar, Depression, Normal, Personality disorder, Stress, Suicidal

Note pour le Groupe 2 : la précision sur *Personality disorder* et *Stress* reste faible (recall $\approx 25\text{--}30\%$). Ces classes sont sous-annotées dans le dataset source. Il est conseillé de traiter les réponses filtrées sur ces deux classes avec prudence, ou d'élargir le seuil de similarité pour compenser.

5 Points de vigilance et perspectives

- **Déséquilibre des classes :** *Depression* est sur-représentée dans `combined_data.csv`, ce qui biaise tous les modèles. Une solution (non implémentée faute de temps) serait le sur-échantillonnage SMOTE ou l'ajustement de `class_weight`.
- **Confusion *Depression* / *Suicidal* :** problème critique dans le contexte médical. La LR avec régularisation L1 (`penalty='l1'`, `solver='saga'`) améliore ce point mais le temps d'entraînement est prohibitif (≈ 4 h). À envisager si plus de ressources de calcul sont disponibles.
- **R² non pertinent :** cette métrique ne doit pas être utilisée pour évaluer des classifieurs multi-classes à labels nominaux. L'accuracy, le F1-score et la matrice de confusion sont les indicateurs à retenir.
- **Amélioration possible :** la Logistic Regression de Logan (LR + TF-IDF, `class_weight='balanced'`) est prometteuse — accuracy de 75.32 % et gains de +0.20 à +0.26 de F1 sur les classes minoritaires — et devrait être envisagée comme version finale si le temps de calcul le permet.